

Assignment Report

Title: **Compiler Principles**

The Tenth Theory Homework

Name	林宏宇
Student ID	23320093
School	计算机学院
Major	计算机科学与技术
Instructor	李文军

[ABSTRACT]

After learning the lecture 10 of compiler principles[1], we got the homework at the end of the lecture.

There are some requirements for the homework:

- The file should be named as **“学号 _ 姓名 _ 第 i 次理论作业”**, where i is the number of the homework.
- The file should be in docx or **pdf** format.
- All the homework should be sent to the TA's email: **406648896@qq.com**
- The deadline for the homework is **24:00 on Sunday of the week** when the homework is assigned.

Keywords: Compiler Principles

1 Paper - Exercise 10.1

Surf the Internet and write a short paper to compare GC on Java platform with GC on Microsoft .Net platform.

1.1 引言

垃圾回收（Garbage Collection，GC）是现代编程语言运行时系统的重要组成部分，它能够自动管理内存，释放不再使用的对象所占用的内存空间，从而降低内存泄漏与程序崩溃的风险。Java 平台与 Microsoft .Net 平台都采用了自动垃圾回收机制，但两者在设计理念、算法实现以及性能优化策略方面存在显著差异。随着云计算、高并发服务以及低延迟应用的发展，垃圾回收性能已经成为衡量运行时系统能力的重要指标。

本文将从代际结构、垃圾回收算法、并发机制、停顿时间控制以及应用场景等方面，对 Java 与.Net 平台的垃圾回收机制进行比较分析。

1.2 Java 平台的垃圾回收机制

代际划分 Java 虚拟机（JVM）采用典型的代际垃圾回收（Generational GC）策略，将堆内存划分为不同区域：

- **新生代（Young Generation）**：用于存放新创建的对象。新生代进一步划分为 Eden 区和两个 Survivor 区（S0、S1）。大多数对象生命周期较短，因此通常会在新生代被快速回收。
- **老年代（Old Generation）**：经过多次 Minor GC 后仍然存活的对象会晋升至老年代。老年代中的对象通常具有较长生命周期。
- **永久代/元空间（Permanent Generation / Metaspace）**：用于存放类元数据、方法信息以及运行时常量池等内容。Java 8 后永久代被元空间（Metaspace）取代。

这种分代设计基于“绝大多数对象朝生夕灭”的经验规律，从而提升垃圾回收效率。

垃圾回收算法 Java 提供了多种垃圾回收器，以适应不同场景需求：

- **Serial GC**：单线程收集器，采用复制算法与标记-整理算法，适用于小型应用或客户端环境。
- **Parallel GC**：通过多线程并行回收提升吞吐量，适用于后台计算与批处理任务。
- **CMS GC（Concurrent Mark Sweep）**：采用并发标记-清除算法，目标是降低停顿时间，但容易产生内存碎片。
- **G1 GC（Garbage First）**：将堆划分为多个 Region，优先回收垃圾最多的区域，在吞吐量与停顿时间之间取得平衡。
- **ZGC/Shenandoah**：Java 新一代低延迟收集器，能够在超大堆内存环境下保持毫秒级停顿时间。

停顿时间特性 Java GC 在执行某些阶段时需要暂停所有应用线程，即“Stop-the-World（STW）”。不同收集器的停顿特性差异明显：

- Serial GC 停顿时间最长，但实现简单；
- Parallel GC 更注重整体吞吐量；
- CMS、G1 与 ZGC 则通过并发标记和并发清理显著降低停顿时间。

因此，Java 在服务器、高并发和低延迟领域具有较强适应能力。

1.3 Microsoft .Net 平台的垃圾回收机制

代际划分 .Net Runtime（CLR）同样采用分代垃圾回收机制，其托管堆（Managed Heap）划分为：

- **第 0 代（Generation 0）**：新创建对象所在区域，回收频率最高。
- **第 1 代（Generation 1）**：作为第 0 代与第 2 代之间的缓冲层。
- **第 2 代（Generation 2）**：存放长期存活对象，回收频率较低。
- **大型对象堆（Large Object Heap，LOH）**：用于存储较大的对象（通常超过 85KB）。

与 Java 相比，.Net 的代际结构更加简洁，但仍然遵循对象生命周期分布规律。

垃圾回收模式 .Net 提供多种 GC 工作模式：

- **工作站模式（Workstation GC）**：主要面向桌面与交互式应用，强调响应速度。
- **服务器模式（Server GC）**：为每个 CPU 核心分配独立堆和回收线程，以提高多核服务器环境下的吞吐量。

- **后台 GC（Background GC）**：允许垃圾回收与应用线程并发执行，从而减少长时间停顿。

停顿时间特性 .Net GC 采用压缩式回收策略，在回收过程中会主动整理内存空间，从而减少碎片化问题。在后台 GC 模式下，CLR 能够在并发阶段完成大部分标记与清理工作，仅在必要时进行短时间暂停。

相较于传统 Java Full GC，.Net 的停顿时间通常更加均匀稳定，更适合交互式应用与企业级桌面系统。

1.4 对比分析

特性	Java	.Net
代数结构	新生代、老年代、元空间	第 0/1/2 代、LOH
主要算法	复制、标记-清除、标记-整理	标记-清除 + 压缩
并发模式	CMS、G1、ZGC 支持并发	Background GC 支持并发
停顿时间	不同收集器差异较大	停顿相对均匀稳定
吞吐量优化	Parallel GC 强调吞吐量	Server GC 强调吞吐量
低延迟支持	ZGC/Shenandoah 支持毫秒级停顿	后台 GC 支持低停顿
配置灵活性	收集器丰富，调优复杂	自动化程度较高
内存碎片处理	部分收集器需额外压缩	默认支持压缩整理

Table 1.1 Java 与.Net 垃圾回收机制对比

1.5 关键差异

- **算法策略差异**：Java 提供大量可选垃圾回收器，开发者能够根据应用场景灵活选择；而.Net 更强调统一化与自动化管理。
- **并发回收能力**：Java 的 G1、ZGC 等收集器通过并发标记显著降低停顿；.Net 的 Background GC 则通过后台并发回收提高响应速度。
- **内存碎片管理**：Java 的 CMS 容易产生碎片，需要额外整理阶段；.Net 默认支持压缩，因此碎片问题相对较轻。
- **调优复杂度**：Java 提供大量 JVM 参数，适合深度性能调优；.Net 的 GC 配置较少，更适合快速开发与企业应用。
- **平台生态差异**：Java 更强调跨平台与服务器部署；.Net 则与 Windows 与 Microsoft 企业生态结合更加紧密。

1.6 应用场景建议

- **Java**：适用于大型分布式系统、云计算平台、互联网服务器以及对吞吐量和扩展性要求较高的场景。通过选择 G1、ZGC 等收集器，可以兼顾性能与低延迟。
- **.Net**：适用于 Windows 企业级应用、桌面交互系统以及需要快速开发与稳定响应的业务系统。Background GC 对实时交互应用具有较好支持。

1.7 结论

Java 与.Net 都采用了现代化的分代垃圾回收机制，并不断向低延迟与高吞吐量方向发展。Java 平台通过丰富的垃圾回收器体系，为不同应用场景提供了高度灵活的性能优化能力；而.Net 平台则通过自动化与压缩式回收机制，在响应时间稳定性方面表现突出。

总体而言，两种平台的垃圾回收机制并不存在绝对优劣，其设计差异更多体现了各自平台的生态定位与工程哲学。随着 Java 的 ZGC/Shenandoah 与.Net 5+ 后台 GC 的持续发展，未来两者在低延迟与大规模内存管理方面仍将不断演进。

References

[1] JUN LI W. Lecture10. principles of compiler construction[Z]. 2026.